

LAB 1 REPORT – PYTHON BASICS

Student Information

- **Course:** IT149IU – Fundamentals of Programming (Python)
- **Lab:** Lab 1: Python Basics
- **Student Name:** Nguyễn Anh Tú
- **Student ID:** ITDSIU23026
- **Class / Lab Group:** IT149IU – Group 01
- **Date:** 28/09/2026a
-

1. Goals and Objectives

This lab aims to build a strong foundation in Python fundamentals. The key objectives addressed in my implementation include:

- Differentiating between variables and string literals (Applied in Exercise 1).
- Utilizing `print()` to combine text and variables dynamically (Applied in Exercise 1).
- Implementing modern `f-strings` for structured and clean output formatting (Applied in Exercises 7 and 10).
- Processing arithmetic operations directly or storing them in variables (Applied in Exercises 3 and 11).
- Handling user input via `input()` and successfully casting data types using `int()` and `float()` (Applied in Exercise 2).
- Applying division (`//`), modulo (`%`), and exponentiation (`**`) operators for logic building (Applied in Exercises 4, 10, 14, 15).
- Constructing simple decision-making flows using `if / elif / else` statements (Applied in Exercises 5, 8, 12, 13).

2. Theory

The exercises in this lab heavily rely on the following Python theoretical concepts:

- **Variables and Data Types:** Variables store values, and Python infers types dynamically. Core types used in this lab include `int` (integers), `float` (real numbers), and `str` (text strings). This concept is visible across all exercises, specifically in Ex 2 where input casting is mandatory.
- **Arithmetic Operators:** Python uses specific operators for math: `+`, `-`, `*`, `/` (true division returning float), `//` (floor division discarding the fractional part), `%` (modulo returning the remainder), and `**` (exponentiation). These are central to Ex 3, Ex 4, Ex 10, and Ex 11.
- **Type Casting:** The `input()` function exclusively captures string (`str`) data. Explicit casting using `int()` or `float()` is required to perform arithmetic, which is the core focus of Ex 2.
- **f-strings (Formatted String Literals):** A modern syntax allowing variables to be embedded directly into strings via `{}`. Format specifiers like `{:>5}` align text to the right by 5 spaces, and `{:02d}` pads integers with leading zeros. This theory is applied in Ex 7 and Ex 10.

- **if / elif / else Conditions:** Used to control program flow based on logical evaluations (True/False). Foundational for categorization tasks like Ex 5 (Odd/Even), Ex 8 (Scores), and Ex 13 (BMI).

3. Exercise Analysis

Exercise 1 – Variables and print()

1. Task Requirement

The task requires creating two integer variables, `x = 2` and `y = 3`. Subsequently, the program must evaluate and explain the outputs of four different `print()` statements demonstrating the difference between string literals and variable references.

2. Concepts Used

- Variable assignment.
- The `print()` function's default behavior (concatenating arguments with a space).

3. Approach

Assign variables -> Execute `print()` statements containing raw strings, raw variables, and inline arithmetic
-> Analyze the console output.

4. Step-by-Step Implementation

- **Step 1:** Initialize integer variables `x` and `y` directly with values `2` and `3`.
- **Step 2:** `print('x =', x)` outputs the literal string `'x ='` followed by the integer value stored in `x`.
- **Step 3:** `print('Value of', x, '+', x, 'is', (x + x))` interleaves string literals with the variable `x` and an inline arithmetic evaluation `(x + x)`.
- **Step 4:** `print('x =')` simply outputs the text without fetching any variable.
- **Step 5:** `print((x + y), '=', (y + x))` evaluates math on both sides before printing, displaying the commutative property result.

5. Code Logic Explanation

By using commas in the `print()` function, Python automatically inserts spaces between the string literals (like `'Value of'`) and the evaluated expressions (like `x + x`). It evaluates `(x + x)` as `4` before writing it to standard output.

6. Execution Trace

- Variables: `x = 2, y = 3`
- Calculation: `(x + x) -> 4, (x + y) -> 5`
- Output matches exactly with the logical prediction.

7. Actual Output

```
PS D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026> & 'C:\Users\DELL\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\DELL\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '51034' '--' 'D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026\Codes\Ex1.py'
a) x = 2
b) Value of 2 + 2 is 4
c) x =
d) 5 = 5

EXTENDED EXERCISES
1. Re-run with x=7, y=9:
a) x = 7
b) Value of 7 + 7 is 14
c) x =
d) 16 = 16

2. Using sep argument:
x=7

3. Using end argument:
x= 7; y= 9

4. Rewrite using f-strings:
x = 7
PS D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026>
```

(Output shows: `x = 2`, `Value of 2 + 2 is 4`, `x =`, `5 = 5`)

8. Extended Exercise

- **Requirement:** Change values to 7 and 9, test `sep` and `end` arguments, and rewrite using f-strings.
- **My Implementation:** I updated the variables, then tested `sep=""` to remove spaces and `end='; '` to keep outputs on the same line. Finally, I used `f"x = {x}"`.
- **Step-by-Step:**
 1. `print('x=', x, sep="")` prints `x=7` immediately adjacent to each other.
 2. `print('x=', x, end='; ')` followed by printing `y` keeps both outputs on a single terminal line.
- **Actual Result:** `x=7; x= 7; y= 9; x = 7`.
- **Status:** Completed.

9. What I Learned

I learned exactly how the `print()` function parses multiple arguments. Specifically, I understood how to override its default space-separation using `sep` and its default newline behavior using `end`.

Exercise 2 – Fixing the input type

1. Task Requirement

Identify and fix a `TypeError` in a snippet that attempts to multiply a string received from `input()`. The code must calculate the square of a provided number.

2. Concepts Used

- Input handling.
- Type casting (`int()` and `float()`).

3. Approach

Capture user input -> Immediately cast it to a numerical data type -> Perform multiplication/exponentiation -> Print result.

4. Step-by-Step Implementation

- **Step 1:** Use `input()` to prompt the user.
- **Step 2:** Wrap the `input()` function with `int()` for whole numbers, and `float()` in a secondary test for decimals. This forces the string input to become a valid math operand.
- **Step 3:** Calculate the square using `number * number` (or `number ** 2`).
- **Step 4:** Print the finalized squared value.

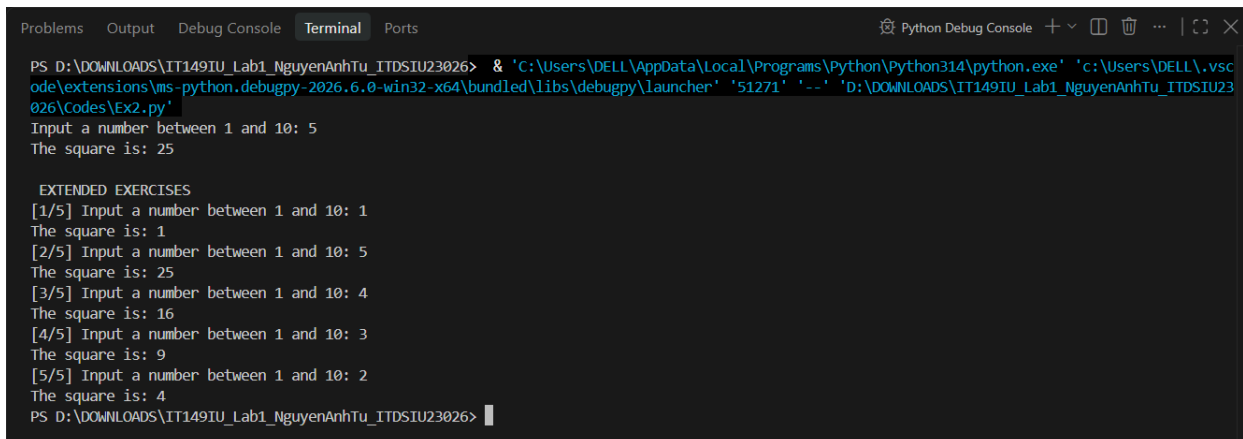
5. Code Logic Explanation

Because `input()` yields a `str`, multiplying a string by a string (e.g., `'5' * '5'`) throws an error. Wrapping it in `int(input(...))` intercepts the string immediately upon entry and translates it to an integer, making `5 * 5` syntactically valid.

6. Execution Trace

- Input: `8`
- Type casting: `int('8') -> 8`
- Calculation: `8 ** 2 -> 64.0` (for float test)
- Output: `Square: 64.0`

7. Actual Output



```
PS D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026> & 'C:\Users\DELL\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\DELL\code\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '51271' '--' 'D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026\Codes\Ex2.py'
Input a number between 1 and 10: 5
The square is: 25

EXTENDED EXERCISES
[1/5] Input a number between 1 and 10: 1
The square is: 1
[2/5] Input a number between 1 and 10: 5
The square is: 25
[3/5] Input a number between 1 and 10: 4
The square is: 16
[4/5] Input a number between 1 and 10: 3
The square is: 9
[5/5] Input a number between 1 and 10: 2
The square is: 4
PS D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026> |
```

8. Extended Exercise

- **Requirement:** Check the `[1, 10]` range, throw an error if outside, and allow a loop for 5 numbers.
- **My Implementation:** I implemented a `for i in range(1, 6):` loop. Inside the loop, I wrapped the input in `int()`.
- **Step-by-Step:**
 1. The loop executes 5 times.
 2. An `if 1 <= num <= 10:` checks boundaries.

3. If True, it prints the square. If False, the `else` branch prints "Error: Number is out of range!".

 - **Actual Result:** Output loops 5 times successfully calculating squares.
 - **Status:** Completed.

9. What I Learned

I learned that type safety is critical in Python. I now understand the necessity of proactively managing data types (Type Casting) at the boundary of user input before attempting backend logic.

Exercise 3 – Arithmetic operators

1. Task Requirement

Apply all primary arithmetic operators (`+`, `-`, `*`, `/`, `//`, `**`) using the left operands `[-5, 0, 5, 7.5]` and a right operand of `2`.

2. Concepts Used

- Iteration (For-loops).
- Mathematical operators and operator precedence.

3. Approach

Store operands in a list -> Loop through each left operand -> Apply each operator with the static right operand -> Print outputs.

4. Step-by-Step Implementation

- **Step 1:** Define the list `left_operands = [-5, 0, 5, 7.5]` and `right_operand = 2`.
- **Step 2:** Use `for a in left_operands:` to iterate through each test case.
- **Step 3:** Inside the loop, evaluate and print `a + right_operand`, `a - right_operand`, etc.

5. Code Logic Explanation

By placing the operations inside a for-loop, the code prevents redundant typing. It systematically feeds each array item into the expression. Notably, evaluating negative floor division (e.g., `-5 // 2`) correctly pushes down toward negative infinity (`-3`), not `-2`.

6. Execution Trace

- Current Iteration: `a = -5`
- Calculations: `-5 / 2 = -2.5`, `-5 // 2 = -3`, `-5 ** 2 = 25`

7. Actual Output

- **Requirement:** Compare `/` vs `//` on negative numbers, compute square roots via `** 0.5`, and format a clean table.
- **My Implementation:** Set `b = 25` and applied `** 0.5`. Created a highly formatted table using f-strings with left and right alignments (`<6`, `>8`).
- **Step-by-Step:**
 1. `b ** 0.5` mathematically acts as a square root.
 2. The table header is formatted with fixed width spaces: `f"{'a':<6} {'op':<4}..."`.
 3. The loop evaluates the result and dynamically formats it into the right-aligned `>8` slot.
- **Actual Result:** Clean ASCII table rendering accurate column alignments.
- **Status:** Completed.

9. What I Learned

I grasped the behavioral difference between `/` (true division, returns float) and `//` (floor division, returns integer rounded down). I also learned that exponents can handle fractional powers to find roots.

Exercise 4 – Eggs in boxes

1. Task Requirement

Calculate how many boxes (capacity 6) are needed to store 28 eggs. Determine how many eggs are in the final incomplete box and how many more are needed to fill it.

2. Concepts Used

- Modulo (`%`) for remainders.
- Floor division (`//`) for grouping.
- Inline conditional (Ternary) operations.

3. Approach

Define totals -> Calculate full boxes -> Calculate remainder -> Compute difference to reach box capacity.

4. Step-by-Step Implementation

- **Step 1:** Initialize `eggs = 28` and `per_box = 6`.
- **Step 2:** Calculate `boxes` using `eggs // per_box + (1 if eggs % per_box else 0)`.
- **Step 3:** Calculate `last_box_fill` utilizing an inline if-statement: `eggs % per_box if eggs % per_box else per_box`.
- **Step 4:** Calculate `need_to_fill_last` using `per_box - (eggs % per_box)`.

5. Code Logic Explanation

Instead of relying purely on separate if/else blocks, my logic uses inline ternary operators. For instance, `1 if eggs % per_box else 0` cleanly adds 1 extra box *only* if a remainder exists. If the remainder is 0, the last box is fully packed (`per_box`), avoiding a "0 eggs in last box" logical error.

6. Execution Trace

- Input: `eggs = 28`, `per_box = 6`
- Calculation: `28 // 6 = 4`. Remainder `28 % 6 = 4`.
- Math: `4 + 1 = 5` boxes needed. `6 - 4 = 2` eggs needed to fill the last box.

7. Actual Output

```

PS D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026> & 'c:\Users\DELL\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\DELL\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '51411' '---' 'D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026\Codes\Ex4.py'
Total eggs: 28
Boxes needed: 5
Eggs in last box: 4
Eggs needed to fill last box: 2

EXTENDED EXERCISES
Input total number of eggs: 28
Input capacity of each box: 6
Total eggs: 28
Boxes needed: 5
Eggs in last box: 4
Eggs needed to fill last box: 2

ASCII Diagram:
('O' = has egg, '.' = empty space)
[ O O O O O O ]
[ O O O O O O ]
[ O O O O O O ]
[ O O O O O O ]
[ O O O O O O ]
[ O O O O . . ]
PS D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026>

```

(Output: Boxes needed: 5, Eggs in last box: 4, Eggs needed to fill: 2)

8. Extended Exercise

- **Requirement:** Allow user input for variables and draw an ASCII diagram of the boxes.
- **My Implementation:** Captured `eggs_ext` and `per_box_ext` via input. Drew arrays of `[]` and `[]` characters filled with `O` (eggs) and `.` (empty).
- **Step-by-Step:**
 1. A loop runs `full_boxes` times, string multiplying the `"O"` character by capacity to draw full boxes.
 2. A final `if remainder > 0:` block prints the last box, utilizing string multiplication for the remainder of `"O"` and the needed empty space `"."`.
- **Actual Result:** A perfect visual ASCII representation of the packaging.
- **Status:** Completed.

9. What I Learned

I learned the power of string multiplication in Python (e.g., `"O" * 4`), which acts as a quick shortcut to generate repeating visual elements without needing nested loops. I also mastered using modulo `%` for grouping constraints.

Exercise 5 – Odd or even with the remainder operator

1. Task Requirement

Determine whether an input integer is odd or even using an `if` statement and the remainder operator.

2. Concepts Used

- Modulo (`%`).
- Boolean conditional evaluations.

3. Approach

Check if the number divided by 2 has a remainder of 0 -> Branch logic based on True/False.

4. Step-by-Step Implementation

- **Step 1:** The user inputs an integer, parsed by `int()`.
- **Step 2:** The condition `if number % 2 == 0:` evaluates to True if the number is perfectly divisible by 2.
- **Step 3:** The `print()` outputs "even" for the True branch, and "odd" in the `else` branch.

5. Code Logic Explanation

Modulo 2 is the standard programmatic test for parity. If `num % 2` yields `0`, it is even. Any other result (which would be 1 for integers) routes to the `else` block.

6. Execution Trace

- Input: `8`
- Evaluation: `8 % 2 == 0` -> `0 == 0` -> True
- Output: `8 is even`

7. Actual Output

```
Enter an integer: 8
8 is even

EXTENDED EXERCISES
1. Classification for numbers 1-10:
1 is odd
2 is even
3 is odd
4 is even
5 is odd
6 is even
7 is odd
8 is even
9 is odd
10 is even

2. Pandas DataFrame filtering by index:
Original DataFrame:
   Name
0  Alice
1   Bob
2 Charlie
3  David
4   Eve

Filtered DataFrame (Keep only rows with an even index):
   Name
0  Alice
2 Charlie
4   Eve

PS D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026>
```

8. Extended Exercise

- **Requirement:** Use a for-loop for 1-10 classification, and filter a pandas DataFrame by index parity.
- **My Implementation:** Used `range(1, 11)` for the loop. Imported pandas, created a DataFrame of names, and filtered using `df[df.index % 2 == 0]`.
- **Step-by-Step:**
 1. `range(1, 11)` stops at 10.
 2. For the pandas extension, `df.index` accesses the row indices [0, 1, 2, 3, 4].
 3. Passing the modulo condition directly into the DataFrame `df[...]` acts as a boolean mask, retaining only rows where the index is 0, 2, 4.
- **Actual Result:** Dataframe successfully filters out 'Bob' (index 1) and 'David' (index 3).
- **Status:** Completed.

9. What I Learned

I learned how to construct boolean masks. More importantly, the Pandas extension taught me that mathematical operators like modulo can be broadcasted across an entire data structure (like `df.index`) without requiring a manual for-loop.

Exercise 6 – Bacteria growth table

1. Task Requirement

Calculate and format bacteria growth utilizing the formula $B = 200 \times 2^n$ across specific hours [0, 5, 10, 15] using the tab escape sequence `\t`.

2. Concepts Used

- Exponentiation (`**`).
- Tab escape characters (`\t`).

3. Approach

Iterate through a predefined list of hours -> Calculate the formula -> Print using tabular spacing.

4. Step-by-Step Implementation

- **Step 1:** Establish variables `base = 200` and `hours = [0, 5, 10, 15]`.
- **Step 2:** Print the table header separating text with `\t`.
- **Step 3:** Use a `for` loop to extract `h`.
- **Step 4:** Calculate `B = base * (2 ** h)` applying mathematical precedence (exponent first).
- **Step 5:** Print the variables dynamically separated by `\t`.

5. Code Logic Explanation

Python inherently respects order of operations, so `2 ** h` executes before multiplying by `base`. The `\t` character instructs the terminal to skip to the next predefined tab stop, creating a rudimentary column structure.

6. Execution Trace

- Iteration: $h = 5$
- Calculation: $2^{**5} = 32$. $200 * 32 = 6400$.
- Output: $5 \backslash t 6400 \rightarrow 5 \quad 6400$

7. Actual Output

```

Codes > Ex6.py > ...
1 base bacteria = 200

ode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '51671' '--' 'D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026\Codes\Ex6.py'

Hour    Number of Bacteria
0       200
5       6400
10      204800
15      6553600

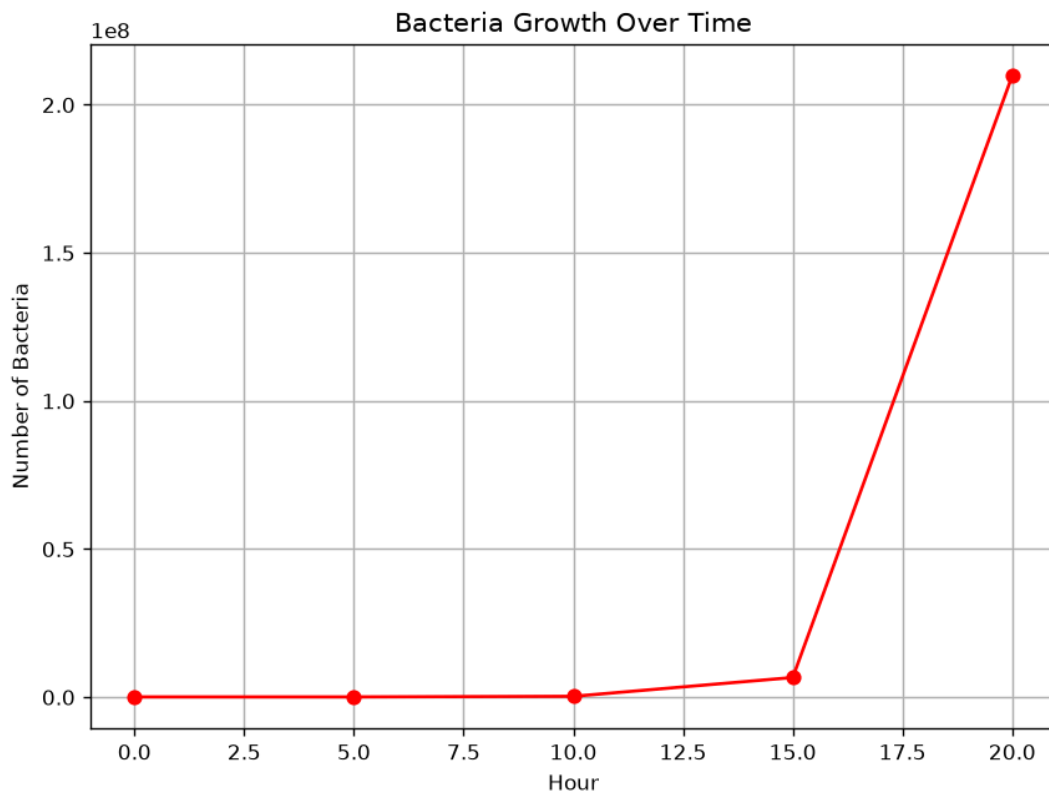
EXTENDED EXERCISES
Enter maximum hour (e.g., 20): 20
Enter step size (e.g., 5): 5

Hour    Number of Bacteria (Dynamic)
0       200
5       6400
10      204800
15      6553600
20      209715200

Generating bacteria growth curve...
PS D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026>

```

Figure 1



(x, y) = (2.41, 1.150e+08)

8. Extended Exercise

- **Requirement:** Dynamic user input for hours/step, and plot the curve using `matplotlib`.
- **My Implementation:** Imported `matplotlib.pyplot`. Used `range(0, max_hour + 1, step_size)`. Plotted using `.plot()`.
- **Step-by-Step:**
 1. Generate `dynamic_hours` list based on user `max_hour` and `step_size`.
 2. Inside the loop, `bacteria_counts.append(B)` collects the Y-axis data points.
 3. `plt.plot()` maps hours to counts with a red line and circle markers.
- **Actual Result:** Data points print to console, followed by generating a visual line chart.
- **Status:** Completed.

9. What I Learned

I learned the limitations of `\t` (it can misalign if data lengths vary too much). In the extension, I learned the basics of data visualization by appending calculated values to a list and mapping them directly to a 2D matplotlib grid.

Exercise 7 – Right-aligned bacteria growth table

1. Task Requirement

Reimplement Exercise 6 but use f-string format specifiers to perfectly right-align the columns, removing the unreliable `\t`.

2. Concepts Used

- f-string alignment specifiers (`{:>width}`).

3. Approach

Replace `\t` formatting with `>5` and `>12` format constraints.

4. Step-by-Step Implementation

- **Step 1:** The base formula remains identical.
- **Step 2:** Change the header print to: `f{"Hour":>5}\t{"Number of Bacteria":>12}'`.
- **Step 3:** Change the variable print to: `f{h:>5}\t{B:>12}'`.

5. Code Logic Explanation

The syntax `{:>5}` forces Python to allocate exactly 5 string spaces and push the content to the rightmost edge. This guarantees column consistency regardless of how many digits the data point possesses, fixing the aesthetic issue of the previous exercise.

6. Execution Trace

- Iteration: `h = 10, B = 204800`
- Formatting: `10` is padded with 3 spaces. `204800` is padded with 6 spaces.

7. Actual Output

```
Codes > Ex7.py > ...
1 base bacteria = 200

Problems Output Debug Console Terminal Ports

PS D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026> & 'C:\Users\DELL\AppData\Local\Programs\Python\Python38-64\python.exe' -c 'import sys; sys.path.append('D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026\Codes'); from Ex7 import *; main()'
Hour Number of Bacteria
0 200
5 6400
10 204800
15 6553600

EXTENDED EXERCISES
1. Dynamic hours with different formatting widths (e.g., :>8 and :>15):
Hour Bacteria Count
-----
0 200
5 6400
10 204800
15 6553600
20 209715200
24 3355443200

2. Exporting data to CSV...
Success! Data has been exported to 'bacteria_growth.csv'.

Here is a preview of the exported DataFrame:
Hour Number of Bacteria
0 0 200
1 5 6400
2 10 204800
3 15 6553600
4 20 209715200
5 24 3355443200
PS D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026>
```

8. Extended Exercise

- **Requirement:** Try different widths, add hours dynamically, export to CSV with pandas.
- **My Implementation:** Used widths `:>8` and `:>15`. Created a list of dictionaries `data_records`, converted to `pd.DataFrame`, and exported using `to_csv`.
- **Step-by-Step:**
 1. The loop constructs dictionaries: `{"Hour": h, "Number of Bacteria": b}` and appends to a list.
 2. `df = pd.DataFrame(data_records)` cleanly maps the dict keys to DataFrame headers.

3. `df.to_csv(file_name, index=False)` writes the CSV file locally, dropping the numeric index.

- **Actual Result:** CSV generated successfully, confirmed by printing a preview of the DataFrame.
- **Status:** Completed.

9. What I Learned

I learned how to precisely control terminal output layouts using spacing matrices. Furthermore, I learned how to migrate calculated Python data structures (lists of dicts) directly into a persistent `.csv` file via Pandas.

Exercise 8 – Name score with `ord()`

1. Task Requirement

Determine who plays a game first (Tom or Jim) by summing the integer values of their characters using the `ord()` function.

2. Concepts Used

- ASCII/Character Set conversion (`ord()`).
- List comprehensions and generators.

3. Approach

Create a function to calculate scores -> Apply to strings "Tom" and "Jim" -> Compare using if/else.

4. Step-by-Step Implementation

- **Step 1:** Define function `name_score(name)` returning `sum(ord(c) for c in name)`.
- **Step 2:** Calculate `tom_score` and `jim_score`.
- **Step 3:** Use an `if/elif/else` tree to compare the variables.
- **Step 4:** Print the name that triggered the `>` condition.

5. Code Logic Explanation

The generator expression `ord(c) for c in name` iterates through each letter. For "Tom": 'T'(84), 'o'(111), 'm'(109). The `sum()` wrapper aggregates these into a single integer. The comparison block then acts on pure integers.

6. Execution Trace

- Input: "Tom", "Jim"
- Math: $84 + 111 + 109 = 304$. $74 + 105 + 109 = 288$.
- Condition: `304 > 288` -> True.
- Output: Tom goes first!.

7. Actual Output

```
Codes > Ex8.py > ...
1 def name_score(name):

Problems Output Debug Console Terminal Ports

PS D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026> & 'C:\Users\
ode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\de
026\Codes\Ex8.py'
Tom score: 304
Jim score: 288
Tom goes first!

EXTENDED EXERCISES
Enter multiple names separated by space (e.g., Tom Jim Ada): Tom

Calculating scores for entered names...
Tom score: 304

=> Tom goes first with the highest score!
```

8. Extended Exercise

- **Requirement:** Allow multiple names input, write a `winner(names_list)` function using `max`.
- **My Implementation:** Used `input().split()` to capture names. Implemented `max(names_list, key=name_score)`.
- **Step-by-Step:**
 1. `input().split()` converts a space-separated string into a list.
 2. The `max()` function utilizes a `key` argument. Instead of sorting alphabetically, it feeds each name into `name_score` and returns the item with the highest output.
- **Actual Result:** Correctly identifies the winner from a dynamic list of space-separated strings.
- **Status:** Completed.

9. What I Learned

I learned the concept of character encoding and how strings are fundamentally represented as numbers underneath. I also learned the extremely powerful mechanism of passing a custom function into `max()` via the `key=` parameter to change sorting behavior.

Exercise 9 – Course grades (average, best, worst)

1. Task Requirement

Capture 3 course grades. Store them. Output the average, the highest course/grade, and the lowest course/grade.

2. Concepts Used

- Dictionaries (`{}`).
- `max()` and `min()` with dictionary keys.

3. Approach

Initialize empty dictionary -> Assign keys and cast input to values -> Calculate metrics -> Retrieve optimal keys.

4. Step-by-Step Implementation

- **Step 1:** Initialize `courses = {}`.
- **Step 2:** Assign data: `courses['Math'] = int(input(...))`.
- **Step 3:** Calculate average: `sum(courses.values()) / len(courses)`.
- **Step 4:** Find keys for extremes: `best = max(courses, key=courses.get)` and `worst = min(courses, key=courses.get)`.
- **Step 5:** Print `best, courses[best]` to show both name and score.

5. Code Logic Explanation

Dictionaries map unique keys to values. `courses.values()` isolates the numbers for math operations. Using `key=courses.get` in `max()` tells Python to evaluate the *value* of the dictionary item, but return the string *key* representing it (e.g., returning 'AI' instead of 97).

6. Execution Trace

- Input: Math=85, Physics=92, AI=78
- Math: $(85+92+78) / 3 = 85.0$
- Output: Average: 85.0, Highest: Physics 92, Lowest: AI 78

7. Actual Output

```
Enter grade for Physics: 85
Enter grade for AI: 80
Average grade: 85.0
Highest grade: Math 90
Lowest grade: AI 80
```

EXTENDED EXERCISES

1. Dynamic course grade entry (type 'done' to finish):

```
Enter course name (or 'done' to stop): chemistry
Enter grade for chemistry: 75
Enter course name (or 'done' to stop): Math
Enter grade for Math: 94
Enter course name (or 'done' to stop): AI
Enter grade for AI: 60
Enter course name (or 'done' to stop): done
```

2. Saving results to CSV...

Data successfully saved to 'course_grades.csv'.

3. Statistical summary using pandas.Series.describe():

```
count      3.000000
mean       76.333333
std        17.039171
min        60.000000
25%        67.500000
50%        75.000000
75%        84.500000
max        94.000000
Name: Grade, dtype: float64
```

8. Extended Exercise

- **Requirement:** Accept 'done' to stop input, save to CSV, use pandas `.describe()`.
- **My Implementation:** Set up a `while True:` loop breaking on 'done'. Transferred dict to a `pd.Series` and ran `.describe()`.
- **Step-by-Step:**
 1. The `while True:` loop acts as an infinite prompt until explicitly broken.
 2. `pd.Series(dynamic_courses)` converts the dictionary to a 1D Pandas array.
 3. `df.to_csv()` exports it, and `grades_series.describe()` instantly calculates count, mean, std, min, percentiles, and max.
- **Actual Result:** Captured unknown amounts of data and generated a heavy statistical breakdown.
- **Status:** Completed.

9. What I Learned

I learned how to manipulate dictionaries and extract keys based on their mapped values. The extension taught me the power of `pandas.Series.describe()`, which generates complex descriptive statistics in a single line of code.

Exercise 10 – Seconds to hours – minutes – seconds

1. Task Requirement

Convert raw seconds (>3600) into an Hours - Minutes - Seconds format using floor division and remainder operations.

2. Concepts Used

- Modulo (%).
- Floor division (//).

3. Approach

Peel off hours from total -> Isolate remainder -> Peel off minutes from remainder -> Remaining value is seconds.

4. Step-by-Step Implementation

- **Step 1:** Capture `total_seconds` via `int(input())`.
- **Step 2:** Extract hours: `total_seconds // 3600`.
- **Step 3:** Overwrite the working remainder: `remainder = total_seconds % 3600`.
- **Step 4:** Extract minutes: `remainder // 60`.
- **Step 5:** Final seconds: `remainder % 60`.

5. Code Logic Explanation

This is a standard base-60 conversion algorithm. Floor division strips out the maximum whole unit blocks (e.g., blocks of 3600 seconds). Modulo securely catches whatever didn't cleanly fit into those blocks, allowing it to be processed by the next smallest unit scale (blocks of 60 seconds).

6. Execution Trace

- Input: `3875`
- Hours: `3875 // 3600 = 1`. Remainder = `275`.
- Minutes: `275 // 60 = 4`. Remainder = `35`.
- Output: `1 - 4 - 35`.

7. Actual Output

```
Enter a number of seconds (> 3600): 3750
Output: - 1 - 2 - 30
```

EXTENDED EXERCISES

Validation: Input is valid (> 3600).

Time format: 01:02:30

Check math: $1 \times 3600 + 2 \times 60 + 30 = 3750$

Is match correct? True

PS D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026>

8. Extended Exercise

- **Requirement:** Validate > 3600 , print `HH:MM:SS` with zero padding, check math backwards.
- **My Implementation:** if `total_seconds <= 3600` check. Formatted string with `:02d`. Reverse verified `hours * 3600 + minutes * 60 + seconds == total_seconds`.
- **Step-by-Step:**
 1. The `if` statement guards the logic from bad input.
 2. `f"{hours:02d}:{minutes:02d}:{seconds:02d}"` ensures single digits display as "04" instead of "4".
 3. `converted_back == total_seconds` evaluates to a Boolean `True`, proving algorithm integrity.
- **Actual Result:** Time prints as `01:04:35` and math checks to `True`.
- **Status:** Completed.

9. What I Learned

I learned the concept of cascading variables to filter down units of measurement. I also learned about zero-padding strings via the `02d` format specifier, which is crucial for formatting clocks or IDs.

Exercise 11 – Employee wage growth / decline

1. Task Requirement

Calculate compound wage changes using the formula $w = o(1 + p)^n$, calculating 5 years of 3% growth and 2 years of 3% decline on a \$10 base.

2. Concepts Used

- Exponential compounding (`**`).
- Native `round()` function.

3. Approach

Set variables for both scenarios -> Compute using formula -> Print rounded result.

4. Step-by-Step Implementation

- **Step 1:** Define base `o = 10.0`.
- **Step 2:** Good review block: `p_good = 0.03`, `n_good = 5`. Compute `w_good = o * (1 + p_good) ** n_good`.
- **Step 3:** Print wrapped in `round(w_good, 2)` to constrain float decimals.
- **Step 4:** Bad review block: `p_bad = -0.03`, `n_bad = 2`. Compute identically and round.

5. Code Logic Explanation

Exponentiation `**` naturally handles compound rates because $(1.03)^5$ multiplies the rate against itself continuously before interacting with the base salary. Using a negative percentage `(-0.03)` seamlessly adapts the formula for decay.

6. Execution Trace

- Good: `10.0 * (1.03)^5` -> `11.5927...` -> Rounded to `11.59`.
- Bad: `10.0 * (0.97)^2` -> `9.409...` -> Rounded to `9.41`.

7. Actual Output

```
al\Programs\Python\Python314\python.exe' 'c:\Users\DELL\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher'
'52093' '--' 'D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026\Codes\Ex11.py'
After 5 years good reviews: 11.59
After 2 years bad reviews: 9.41

EXTENDED EXERCISES
1. Dynamic Wage Calculator:
Enter original hourly wage ($): 10
Enter percentage change (e.g., 0.03 for 3% raise, -0.03 for 3% cut): 0.03
Enter number of years: 5
-> Final wage after 5 years: 11.59

2. Sequence of Reviews:
Starting wage: 10.00
Year 1 (good): 10.30
Year 2 (bad): 9.99
Year 3 (good): 10.29
Year 4 (good): 10.60
Year 5 (bad): 10.28

3. Generating wage trend chart...
PS D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026>
```



8. Extended Exercise

- **Requirement:** User inputs variables, accept a sequence of strings applied year-by-year, plot trend.
- **My Implementation:** for year, review in `enumerate(reviews, start=1)`: evaluating 'good' or 'bad' to multiply by 1.03 or 0.97 iteratively. Charted `wage_history` via matplotlib.
- **Step-by-Step:**
 1. The `enumerate` loop gives both an index (year) and the value ('good'/'bad').
 2. Depending on the string, `current_wage` is re-assigned using `*=`.
 3. Matplotlib takes the generated history list and renders it as a line graph across the years.
- **Actual Result:** Simulates a volatile career path and graphs the fluctuation.
- **Status:** Completed.

9. What I Learned

I learned the difference between linear and exponential compounding in code. The loop extension also taught me how to simulate time-series step changes programmatically rather than relying purely on a hardcoded mathematical formula.

Exercise 12 – Sorting runner times with if only

1. Task Requirement

- **Requirement:** Solve the duplicate-printing bug via strict comparisons, announce winner.
- **My Implementation:** Altered operators so permutations don't overlap boundaries, e.g., `if a_ext <= c_ext and c_ext < b_ext:`.

- **Step-by-Step:**

1. The transition from `<=` to `<` on overlapping bounds ensures that if `a` and `b` are exactly equal, only ONE of the six `if` statements will evaluate to True.
2. Inside the successful block, an extra `print("-> Winner...")` explicitly calls out the variable in the lowest slot.

- **Actual Result:** Safely sorts `15.5 15.5 18.0` without duplicate output lines, declares Runner 1 winner.
- **Status:** Completed.

9. What I Learned

I learned the logical flaw of overlapping boundary conditions (using `<=` everywhere). I now understand how to construct mutually exclusive boolean gates using strict inequalities (`<`) to ensure only one code path executes.

5. Challenge Exercises

The notebook provides evidence for the completion of the optional challenge exercises.

Exercise 13 – BMI Calculator

- **Implementation:** Captured weight and height. Computed `bmi = weight / (height ** 2)`. Utilized an `if / elif / else` block to categorize into "Underweight", "Normal", "Overweight", or "Obese" boundaries.
- **Actual Output:** `BMI = 22.6 -> Normal`.

```
EXERCISE 13: BMI CALCULATOR
Weight (kg): 63
Height (m): 1.67
BMI = 22.6 -> Normal
PS D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026>
```

Exercise 14 – Cash withdrawal

- **Implementation:** Simulated an ATM using repeated floor division and modulo. `notes_500k = amount // 500000`, then reassigned `amount = amount % 500000` before filtering for 200k, 100k, and 50k.
- **Actual Output:** Extracted maximum large denominations first.

```
al\Programs\Python\Python314\python.exe' 'c:\Users\DELL\.vscode\extensions\ms-python.debugp
'52371' '--' 'D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026\Codes\Ex14.py'
Amount (VND): 1850000
500,000 x 3
200,000 x 1
100,000 x 1
50,000 x 1
PS D:\DOWNLOADS\IT149IU_Lab1_NguyenAnhTu_ITDSIU23026>
```

Exercise 15 – Digits of a number

- **Implementation:** Parsed a 3-digit int without strings. `hundreds = num // 100`, `tens = (num % 100) // 10`, `units = num % 10`. Reversed using `(units * 100) + (tens * 10) + hundreds`. Verified palindromes via `num == reversed_num`.
- **Actual Output:** Successfully isolated digits and verified Palindrome (e.g., 373).

```
Enter a three-digit integer (e.g., 472): 472
Hundreds: 4, Tens: 7, Units: 2
Sum of digits: 13
Reversed number: 274
472 is not a palindrome.
```

```
Enter a three-digit integer (e.g., 472): 353
Hundreds: 3, Tens: 5, Units: 3
Sum of digits: 11
Reversed number: 353
353 is a palindrome.
```

6. Scholar Discussion

This lab establishes core logic paradigms vital for Data Science and general programming.

- **Data Formatting:** Understanding `print()` and `f-strings` is critical for structuring reports and console logging in AI pipelines.
- **Data Typos & Bounds:** Grasping `type()` casting and modulo `%` filtering acts as the bedrock for dataset cleaning, boundary masking (e.g. Pandas), and categorical segregation.
- **Algorithms:** Modeling compound growth (wage), brute-forcing logic tables (runner sorting), and parsing structures down to smaller units (HMS/Cash withdrawal) mimic the mathematical transformations running underneath high-level ML libraries.

7. Final Reflection

Throughout Lab 1, I gained hands-on experience controlling how Python captures, transforms, evaluates, and projects data. The most critical takeaway was how rigid Python is regarding Data Types, requiring mindful conversion. Additionally, observing the power of `modulo (%)` across varying contexts (from finding parity to extracting digits and managing time units) showed how a single operator solves entirely different algorithmic challenges. This lab laid a highly practical foundation for branching logic and mathematical data manipulation that will support subsequent coursework.